

BORDER OPTICS — POORI RESEARCH SAMJHO

PDF 7 of 22 — Acquisition: Control Group Selection select_control_villages.py — Sabse Complex File, Poori Line By Line

Sakshi ke liye, Sakshi ki apni research par. Ye file akeli itni complex hai ki ise apni poori PDF mili hai — koi doosri file iske saath share nahi ho rahi. Har line, do baar explain: pehle uska research-purpose, fir uska Python-syntax. Aur is file ki history sabse lambi hai — char alag rounds mein badli hai, har round mein ek real bug pakda gaya aur fix hua.

Ye file BORDER OPTICS ka *control group* banati hai — 721 non-VVP villages jo treated (VVP-I) villages se compare kiye jaate hain. PDF 4 mein jo H4 DiD test dekha tha, uska poora data yahin se aata hai. Is file ki kahani seedhi nahi hai: 753 se shuru hui, fir 735 hui, fir 732, aur akhir mein 721 — har number-change ke peeche ek genuine, documented bug tha jo pakda aur fix kiya gaya.

Shuru Karte Hain — Control Group Ka Poora Matlab

Ab tak humne jo files dekhi hain — geocoding (PDF 6) — wo treated villages ke liye coordinates dhundti hain. Lekin PDF 4 mein humne dekha tha ki sirf treated villages ka before/after dekhna kaafi nahi hai: agar ek gaon 2021 se 2025 ke beech badla, ye kaise pata chale ki ye VVP-I scheme ki wajah se badla, ya ye badlaav poore region mein ho raha tha, VVP-I ho ya na ho? Iska jawaab hai ek control group — non-VVP villages jo treated villages jaisi hi jagah par hain, lekin scheme ka hissa nahi hain. Agar treated aur control dono ek jaisa badlaav dikhaein, toh ye area ka general trend hai. Agar sirf treated badle, tab VVP-I ka asli effect maana ja sakta hai. Isi comparison ko Difference-in-Differences, ya DiD, kehte hain.

EXAMPLE — Do Bachon Ki Height Ki Analogy

Socho tu ek bachhe ko ek special diet par rakhti hai, ek saal tak, ye dekhne ke liye ki kya wo diet uski height badhata hai. Saal ke baad, bachha 5cm lamba ho gaya — lekin kya ye diet ki wajah se hua, ya bachha waise hi badhta (growth ki natural process)? Iska jawaab akele us ek bachhe se nahi milega. Tujhe ek doosra, similar bachha chahiye — same umar, same family background — jo wahi diet nahi le raha. Agar wo bachha bhi 5cm badha, toh diet ka koi khaas effect nahi tha, ye toh natural growth thi. Agar wo bachha sirf 2cm badha, toh diet ka genuine +3cm ka effect dikhta hai. Control group exactly yahi kaam karta hai — VVP-I ke 'natural growth' ko compare karne ke liye ek similar, lekin scheme-se-bahar ka gaon chahiye.

Ye file, `select_control_villages.py`, wahi 'doosra bachha' dhundti hai — har treated village ke district mein, OpenStreetMap se non-VVP villages/hamlets khoji jaati hain, unme se jo treated villages jaise dikhte hain (same district, border se similar doori) unhe select kiya jaata hai, aur jo treated village ka hi duplicate nikle (alag naam se, ya galat tarike se) unhe carefully exclude kiya jaata hai. Ye 'carefully exclude karna' hissa hi is file ko sabse complex banata hai — jaisa hum aage dekhenge, isi hisse mein sabse zyada real bugs mile aur fix hue.

Poori History Ek Nazar Mein — 753 Se 721 Tak

Is file ka control-village-count chaar baar badla, aur har baar ek specific, documented reason se. Neeche ek table hai jo पूरी lineage dikhata hai — is PDF mein hum har row ki kahani, uske sambandhit code ke saath, detail mein padhenge.

Count	Kab Aaya	Kyun Badla
753	Pehli baar (Entry 13)	Fresh run, retry/backoff ke saath, sab 14 districts
735	Entry 15 (Entry 14 ke fix se)	18 cross-district duplicate OSM points hataye

732	Entry 25 (Entry 24 ke fix se)	20 treated-duplicate + 3 official-name contamination hatayi
721	Entry 37	District-boundary verification — 2 districts ke bbox-only matches saaf hue

Ek baat dhyan rakhna: ye koi 'file kharab thi, baar-baar kharab hoti gayi' wali kahani nahi hai. Ye ulta hai — har round mein file *zyada accurate* hoti gayi, kyunki har round mein ek genuine gap pakda gaya jo pehle kisi ne notice nahi kiya tha. Ye BORDER OPTICS ke poore project ka ek core principle hai, jo is file mein sabse zyada saaf dikhta hai: accuracy ko pehle rakho, chahe usse purane, already-reported numbers badalne pade.

Ab, Poori File — Line By Line

Docstring — File Apni Kahani Khud Bata Rahi Hai

```
"""
```

```
Builds the non-VVP control group: matched on district (verified against the
real Nominatim boundary, not just the Overpass bbox), soft filter at 1.5x
treated max border-distance, capped at 3x treated count per district.
```

```
Fixed two ways a "control" could secretly be a treated village: (1) distance
filter now uses the same geodesic method as compute_border_distance.py
instead of a distorted single UTM zone, (2) exclusion now also catches
same-village-different-script duplicates (coord proximity check) and official
VVP-I villages that never got geocoded, not just exact name matches.
```

```
Delete data/processed/border_optics_control_villages.csv before rerunning
after this fix, or resume logic thinks old (contaminated) rows are still good.
```

```
"""
```

Purpose: Ye docstring kisi normal file ke docstring jaisa nahi hai — ye khud ek chhota history chapter hai. Pehla paragraph batata hai file abhi kya karti hai (district-match, boundary-verified, soft distance-filter, per-district cap). Doosra paragraph seedha batata hai ki file mein do specific fixes hue hain — aur kis tarah ka bug wo fix karte hain (ek 'control' village secretly treated village nikalna). Teesra paragraph ek operational warning hai — purani output file delete kiye bina rerun karna, resume-logic ko confuse kar dega. Syntax: Triple-quoted ("""...""") module docstring, jaisa PDF 6 ki files mein dekha tha. Is baar ye lamba hai — teen paragraphs, blank lines se separated — kyunki is file ki history normal se zyada rich hai.

EXAMPLE — Ek Docstring Jo Khud History Record Karta Hai — Kyun Ye Achhi Practice Hai

Development Log mein poori history detail mein hai (Entries 23, 24, 25 — jo hum is PDF mein aage dekhenge), lekin file ka apna docstring bhi ek chhota summary rakhta hai. Ye isliye zaroori hai kyunki agar koi (khud Sakshi bhi, mahino baad) sirf is ek file ko khole, bina poore Development Log ko padhe, use turant pata chal jaaye ki ye file 'v1 se seedhi' nahi hai — iske peeche real fixes hain, aur ek specific operational step (purani file delete karna) zaroori hai warna galat results aa sakte hain. Achha code sirf 'kaam karna' nahi hota — wo apni khud ki history bhi carry karta hai, taaki future mein koi accidentally purane bug ko wapas na le aaye.

Imports

```
import time
import unicodedata

import geopandas as gpd
import numpy as np
import pandas as pd
import requests
from pyproj import Geod
from shapely.geometry import Point, shape
from shapely.ops import nearest_points
```

Purpose: Ye file ab tak ki sabse zyada libraries import karti hai — kyunki iska kaam sabse zyada alag-alag cheezon ko jodta hai: internet se data mangwana, geographic shapes ke saath kaam karna, aur precise distance nikaalna. time wait/delay ke liye. unicodedata naam ko normalize karne ke liye (aage `normalize_name()` mein dikhega). geopandas boundary shapefile padhne ke liye. numpy fast array-math ke liye. pandas table operations ke liye. requests Overpass aur Nominatim se baat karne ke liye. pyproj se Geod — precise geodesic (Earth ki curved surface par) distance calculate karne ke liye. shapely se Point (ek single coordinate), shape (GeoJSON ko geometry object mein badalna), aur nearest_points (do geometries ke beech sabse paas ka point pair dhundna). Syntax: `import X` aur `import X as Y` pehle bhi dekh chuke hain. `from module import specific_thing` pattern yahan teen jagah dikhta hai — poora pyproj ya shapely module import karne ke bajaye, sirf wo specific classes/functions liye ja rahe hain jo is file ko chahiye. Beech mein ek blank line hai — ye sirf readability ke liye hai (standard library imports upar, third-party libraries niche), Python ko iski zaroorat nahi hai.

EXAMPLE — Geodesic Distance Kya Hoti Hai, Aur Ye File Isse Itni Seriously Kyun Leti Hai

Zameen gol hai, flat nahi. Agar tu do coordinates ke beech ki doori seedhe Pythagoras formula (jaise ek flat map par ruler se naapna) se nikaale, ye thoda galat aayega — kitna galat, ye tere location aur doori par depend karta hai. `pyproj.Geod` Earth ke asli, ellipsoid shape (poora gol nahi, thoda chapta) ko account mein leke distance nikaalta hai — ye 'geodesic' distance kehlaati hai, aur ye hamesha zyada accurate hoti hai ek flat-map calculation se. Is file mein ye itna zaroori isliye hai kyunki iski ek purani version mein exactly ye galti thi — flat-map (UTM projection) distance use ho rahi thi, jo Arunachal Pradesh jaisi jagah par kaafi off ho gayi thi. Iski poori kahani aage, GEOD constant ke paas, milegi.

Path Constants — Kahan Se Padhna, Kahan Likhna

```
TREATED_PATH = "data/processed/border_optics_master_villages_with_distance.csv"
BOUNDARY_PATH = "data/raw/ne_10m_admin_0_boundary_lines_land/ne_10m_admin_0_
boundary_lines_land.shp"
OUT_PATH = "data/processed/border_optics_control_villages.csv"
```

Purpose: Teen file-paths ek hi jagah define ho rahe hain. TREATED_PATH wahi 251 core-sample treated villages ki file hai jisme unka border-se-distance bhi already calculate ho chuka hai (PDF 4/5 mein dekha tha). BOUNDARY_PATH Natural Earth ki official country-boundary shapefile hai — India ki asli border-line ka data. OUT_PATH wahi file hai jahan final control-village list save hogi — yahi file hai jo baar-baar 753 se 721 tak badli. **Syntax:** Teen simple string assignments, sab module-level. In naamo ka UPPERCASE hona (Python convention) batata hai ki inhe 'constant' ki tarah treat karna hai — file ke andar kahin bhi ye values change nahi hongii.

OFFICIAL_PRIORITY_VILLAGE_PATHS — Teen States Ki Poori Official Lists

```
OFFICIAL_PRIORITY_VILLAGE_PATHS = {
    "Arunachal Pradesh": "data/raw/arunachal_pradesh_vvp_villages.csv",
    "Sikkim": "data/raw/sikkim_vvp_villages.csv",
    "Uttarakhand": "data/raw/uttarakhand_vvp_villages.csv",
}
```

Purpose: Ye dictionary teen core states ki *raw*, *poori* official VVP-I village lists ka path rakhta hai — na ki sirf jo geocode ho paayi (251), balki jo bhi official gazette mein 'priority village' ki tarah list hui thi, chahe wo geocode hui ho ya nahi. Himachal Pradesh yahan jaan-bujh kar nahi hai — jaisa pehli files mein bhi dekha, Himachal sirf illustrative case study hai, core sample ka hissa nahi. Ye dictionary kyun zaroori hai, iski पूरी कहानी aage load_official_priority_village_names() function ke paas milegi — ye ek real data-gap se seedha juda hai jo Development Log Entry 23 mein mila. **Syntax:** Dictionary literal, teen entries — jaisa geocode_villages.py ke STATE_FILES mein dekha tha, wahi pattern.

COORD_DUPLICATE_THRESHOLD_M — Kitni Paas Ho Toh 'Wahi Gaon' Maana Jaaye

```
# within this distance of a treated village = same settlement, not a real control  
COORD_DUPLICATE_THRESHOLD_M = 50
```

```
GEOD = Geod(ellps="WGS84")
```

Purpose: Pehli line ek constant define karti hai: agar koi candidate control-village, kisi bhi treated village ke 50 meter ke andar hai, use 'wahi physical settlement' maana jaayega, alag naam ke bawajood — ek genuine control nahi. Doosri line ek Geod object banati hai, WGS84 ellipsoid model ke saath (yahi standard model hai jo GPS bhi use karta hai) — ye poori file mein har jagah geodesic distance nikaalne ke liye reuse hota hai. Syntax: `COORD_DUPLICATE_THRESHOLD_M = 50` ek simple integer constant hai. `Geod(ellps="WGS84")` pyproj library ka ek class-instantiation hai — `ellps="WGS84"` parameter batata hai kaunsa Earth-shape-model use karna hai (WGS84 sabse common hai, GPS-grade accuracy ke liye). Ye GEOD object poori file mein baar-baar `GEOD.inv(...)` ki tarah call hoga — ek baar banaya, kayi jagah reuse kiya.

EXAMPLE — Ye Line Kyun Yahan Hai — 20 Gaon Jo Do Baar Gine Gaye

Development Log Entry 23 mein ek bada, real discovery hai: 735-village control list mein se 20 villages, physically, treated villages ke *exact same coordinate* par baithe the — sirf alag naam ke saath. Ye koi coincidence nahi tha, ye ek hi OSM point tha jo do baar, do alag naamo se, dataset mein aa gaya tha: transliteration variants jaise Sibia/Sebia ya Nisuk/Nissuk, punctuation ke differences jaise Lengdi/Liang vs Lengdi Liang, aur sabse tricky — OSM ka alternate-language naam tag ek bilkul alag script mein: Gunji ka apna Devanagari-script version (isi font mein render nahi hota, isliye yahan nahi dikhaya ja raha, lekin Development Log mein maujood hai), aur Chate ka apna Chinese-script version, bilkul waise hi. Purana exclusion sirf naam compare karta tha — aur ye pairs kabhi bhi naam se match nahi karenge, kyunki unke naam hi alag hain. Isi gap ko band karne ke liye ye 50-meter coordinate-proximity check add hua — naam ki jagah, seedhe location compare karo. Iski poori kahani, discovery se fix tak, hum `nearest_treated_distance_m()` function ke paas dekhenge.

Overpass Settings — Free, Public OSM Query Service Se Baat Karna

```
OVERPASS_URL = "https://overpass-api.de/api/interpreter"
OVERPASS_TIMEOUT_S = 90
REQUEST_PAUSE_S = 2.0 # be polite to the shared public Overpass instance
```

Purpose: Overpass ek free, public service hai jisse OpenStreetMap ka raw data query kiya ja sakta hai — 'is bounding-box ke andar sab villages/hamlets do' jaisi queries. `OVERPASS_TIMEOUT_S = 90` batata hai ki ek query ko jawaab dene mein zyada se zyada 90 second lag sakte hain (ye Nominatim ke 20-second timeout se kaafi zyada hai, kyunki Overpass queries — poore district ka data — bhaari hoti hain). `REQUEST_PAUSE_S = 2.0` har successful query ke baad 2 second ka wait hai, taaki is shared, free server ko overload na karein. Syntax: Teen constants: ek string (URL), ek integer (timeout seconds), ek float (pause seconds, decimal allowed). Comment (# be polite...) code ka hissa nahi hai, sirf ek human-readable note hai kyun ye pause zaroori hai.

EXAMPLE — Overpass Kyun Itni Careful Handling Maangta Hai — Entry 13 Ki Pehli Live Run

Development Log Entry 13 mein likha hai: jab ye script pehli baar live chalayi gayi, Overpass ka shared public instance rate-limiting hard maar raha tha — 429 (too many requests) aur 504 (timeout) errors, khaaskar bade district bounding-boxes ki queries par. Isi experience ne retry/backoff logic (jo hum aage dekhenge) aur per-district checkpointing ko zaroori banaya. Ek free, shared server istemal karte waqt, apni script ko 'polite' rakhna sirf achhi practice nahi hai — ye zaroori hai, warna server request hi reject karna shuru kar deta hai.

Nominatim Settings — Ye Naya Hai (Entry 37 Mein Add Hua)

```
NOMINATIM_URL = "https://nominatim.openstreetmap.org/search"
NOMINATIM_PAUSE_S = 1.0 # Nominatim's usage policy asks for max 1 req/sec
```

Purpose: Ye do lines PDF 6 ke `geocode_villages.py` mein bhi dekhi thi — Nominatim ka address geocoding API. Lekin is file mein Nominatim ka istemal alag hai: yahan gaanv dhundne ke liye nahi, balki poore *district ki boundary polygon* dhundne ke liye use hota hai (aage `fetch_district_polygon()` mein dikhega). `NOMINATIM_PAUSE_S = 1.0` Nominatim ki apni usage-policy follow karta hai — maximum ek request per second. Syntax: String aur float constant, jaisa upar dekha. Comment mein Nominatim's jaisa apostrophe-wala shabd hai — Python string ke andar single-quote character seedha likh sakte hain agar poora string double-quotes se shuru-khatam ho raha ho, jaisa yahan hai.

HEADERS Aur Retry Constants

```
# Overpass rejects requests with no User-Agent (406 Not Acceptable) -- same
# identifying header geocode_villages.py already sends to Nominatim
HEADERS = {"User-Agent": "border_optics_research_sakshi_maske (contact:
```

```
sakshimaske303@gmail.com)"]}
```

```
# Overpass rate-limits (429) / times out (504) under load -- 5 retries with  
# 20/40/80/160s backoff caps a stuck district at ~5 min before moving on.  
MAX_RETRIES = 5  
RETRY_WAIT_SECONDS = 20  
RETRYABLE_STATUS_CODES = {429, 502, 503, 504}
```

Purpose: HEADERS wahi identifying User-Agent hai jo PDF 6 mein geocode_villages.py Nominatim ko bhejta tha — yahan dono, Overpass aur Nominatim dono, ko bheja jaata hai, kyunki Overpass bhi bina User-Agent ke request reject kar deta hai (406 error). Retry constants define karte hain: 5 baar tak retry karo, har retry se pehle 20 second wait karo — lekin comment mein '20/40/80/160s backoff' likha hai, matlab wait time har retry ke saath double hota jaata hai (isko 'exponential backoff' kehte hain, aage code mein exact formula dikhega). RETRYABLE_STATUS_CODES ek set hai un HTTP status-codes ka jinke liye retry karna sensible hai — 429 (too many requests), 502/503/504 (server temporarily down/overloaded) — in sab mein thodi der ruk ke phir try karna kaam kar sakta hai, isliye ye 'retryable' hain. Syntax: Dictionary literal for HEADERS (ek hi key-value pair). Teen aur constants: do integers, ek set literal ({429, 502, 503, 504} — curly braces mein comma-separated numbers, koi key-value pair nahi, isliye Python ise set samajhta hai na ki dictionary).

Distance Aur Cap Multipliers — Control Group Ka Design-Philosophy

```
MAX_DISTANCE_MULTIPLIER = 1.5 # candidate cap: up to 1.5x the treated sample's max  
distance  
MAX_CANDIDATES_PER_DISTRICT_MULTIPLIER = 3 # cap control count at 3x treated  
count per district
```

Purpose: Ye do numbers control-group ke design ka core hain. Pehla: ek candidate control-village tabhi eligible hai jab uski border-se-doori, us district ke treated villages ki *sabse zyada* doori se 1.5 guna se zyada na ho — matlab, control villages bhi roughly usi 'border-area character' ke hone chahiye jaise treated villages, sirf distant lowland towns nahi. Doosra: har district mein, control villages ki ginti treated villages ki ginti se 3 guna se zyada nahi hogi — ye ek reasonable-size sample banaye rakhta hai, bina district ko ek hi tarah ke sainkdon candidates se bhar diye. Syntax: Do float/int constants, comments ke saath jo unka exact role batate hain.

EXAMPLE — District-Matching Vs Tight-Distance-Matching — Ek Design Choice Jo Jaan-Bujh Kar Li Gayi

Development Log Entry 12 mein is poore design ka reasoning likha hai. Ek option ye hota ki control villages ko treated villages ke *ekdum* same distance-band mein match kiya jaaye (jaise, sirf +/- 2km ke andar). Lekin isse do problems hoti: pehla, candidate pool itna chhota ho jaata ki kaafi districts mein control villages milte hi nahi. Doosra — aur ye zyada subtle point hai — jo villages is tight band mein survive karte, wo

mostly wahi hote jo treated village ke bilkul bagal mein hain, jo shayad kisi specific reason se hi VVP-I list se bahar reh gaye hon (matlab, wo bhi ek tarah se 'treated jaise hi' hain, ek clean comparison nahi denge). Isliye district-level matching (same district, generous 1.5x distance-cap) chuna gaya — 'same regional trend' wala comparison defensible rehta hai, aur DiD design ko ek genuine, usable sample bhi milta hai.

normalize_name() — Naam Ko Ek Consistent Form Mein Laana

```
def normalize_name(name):  
    name = unicodedata.normalize("NFKD", str(name)).encode("ascii", "ignore").decode()  
    return " ".join(name.lower().split())
```

Purpose: Ye function kisi bhi gaon ke naam ko ek consistent, comparable form mein badalta hai — taaki 'Sibia' aur 'sibia ' (extra space ke saath) same maane jaayein. Ye poori file mein har jagah naam-comparison ke liye use hota hai. Syntax: str(name) safety ke liye string mein convert karta hai. unicodedata.normalize("NFKD", ...) Unicode characters ko unke 'decomposed' form mein todta hai — jaise ek accented character (é) ko do parts mein (e + accent-mark) todna. .encode("ascii", "ignore") fir sirf plain-ASCII characters rakhta hai, baaki sab (accents, non-Latin scripts) *chhod* deta hai — "ignore" ka matlab hai jo encode nahi ho sakta, use silently drop karo, error mat do. .decode() wapas bytes se normal string mein le aata hai. Last line: .lower() sab lowercase, .split() whitespace par todta hai extra spaces hatake, ".join(...)" wapas single-space se jodta hai.

EXAMPLE — Gunji Ki Devanagari Spelling — Ye Function Structurally Kyun Ye Pakad Hi Nahi Sakta

Yahi function hai jiski limitation ne Entry 23 ki 20-duplicate discovery ko zaroori banaya. Socho ek treated village ka naam 'Gunji' hai, aur OSM mein wahi exact physical point ki uski Devanagari-script spelling bhi alt-name tag mein hai (asal Development Log mein us Devanagari spelling ka poora example hai; yahan is PDF ke font mein wo script render nahi hoti, isliye sirf iska matlab describe kiya ja raha hai). normalize_name("Gunji") return karega "gunji" — saaf ASCII text. Lekin us Devanagari spelling par normalize_name(...) chalane se kya hoga? Devanagari characters NFKD ke baad bhi ASCII nahi hain, toh .encode("ascii", "ignore") unhe poori tarah drop kar dega — result ek *khaali string* hoga, "". Ab "gunji" == ""? Bilkul nahi. Ye function, apne design se hi, kabhi ye pair match nahi kar sakta — kyunki inke normalized forms mein koi shared text hai hi nahi. Yahi wajah hai ki ek naam-based filter, chahe kitna bhi smart normalize kyun na kare, ek same-point-different-script duplicate ko kabhi nahi pakad sakta — usko pakadne ke liye coordinate compare karna padta hai, naam nahi. Ye exact insight hai jisne nearest_treated_distance_m() function ko zaroori banaya, jo hum aage dekhenge.

load_official_priority_village_names() — Poori Official List, Na Sirf Geocoded Waali

```
def load_official_priority_village_names():
    names_by_state = {}
    for state, path in OFFICIAL_PRIORITY_VILLAGE_PATHS.items():
        try:
            names_by_state[state] = set(normalize_name(n) for n in
pd.read_csv(path)["Habitation"])
        except FileNotFoundError:
            print(f" WARNING: {path} not found -- official priority-village name check "
                f"for {state} will be skipped this run.")
            names_by_state[state] = set()
    return names_by_state
```

Purpose: Ye function OFFICIAL_PRIORITY_VILLAGE_PATHS ki teen raw CSVs padhta hai — har state ki *poori* official VVP-I priority-village list, na ki sirf wo 251 jo successfully geocode ho paayi. Har state ke liye, saare habitation-naam normalize karke ek set mein daal diye jaate hain. Agar koi file na mile (missing), crash nahi hota — ek warning print hoti hai aur us state ke liye khaali set use hota hai, taaki baaki states ka processing rukein nahi. Syntax: Khaali dictionary {} se shuru. for state, path initems(): dictionary ke pairs par loop. try/except FileNotFoundError — sirf 'file missing' error ko specifically pakadta hai, koi aur error ho toh crash hoga (jo sahi hai, kyunki koi aur error genuinely unexpected hoga). set(normalize_name(n) for n in pd.read_csv(path)["Habitation"]) ek *generator expression* hai set(...) ke andar — 'Habitation' column ke har naam ko normalize karo, aur sabko ek set mein daal do (set automatically duplicates hata deta hai).

EXAMPLE — 301 Names Jo Kabhi Check Hi Nahi Hue The — Entry 23 Ki Doosri Discovery

Development Log Entry 23 mein ek doosra, alag gap mila — pehle wale (coordinate duplicates) se bilkul alag reason ka. Purana exclusion logic sirf treated_names se compare karta tha, jo border_optics_master_villages_with_distance.csv se banta hai — aur ye file sirf wo 251 villages rakhti hai jo *successfully geocode* ho paayi. Lekin teeno states ki raw official lists mein kul milaake 552 priority-village naam hain (454 Arunachal + 46 Sikkim + 51 Uttarakhand) — matlab 301 official naam kabhi treated_names mein aaye hi nahi, kyunki wo geocode fail ho gaye the. In 301 mein se, 735-village control list ko check karne par, teen exact matches mile: Gai (Anjaw district), Tayeng (Kurung Kumey district), aur Pamdung (Tawang district) — teeno officially VVP-I list mein the, lekin geocode nahi ho paaye the, isliye purana check unhe treated maanta hi nahi tha, aur wo apne asli, VVP-listed naam ke saath hi control group mein reh gaye. Yahi is function ka poora purpose hai — sirf 251 ke bajaye, poori 552 ki list se compare karna, taaki ye teen jaisi galtiyan dobara na hon.

nearest_treated_distance_m() — Coordinate Se Duplicate Pakadna

```
def nearest_treated_distance_m(lat, lon, treated_lats, treated_lons):
    n = len(treated_lats)
    if n == 0:
        return float("inf")
    _, _, dists_m = GEOD.inv(
        np.full(n, lon), np.full(n, lat), treated_lons, treated_lats,
    )
    return float(np.min(dists_m))
```

Purpose: Ye function ek candidate control-village ke coordinate ko, saare treated villages ke coordinates se ek saath compare karta hai, aur sabse kam doori wapas bhejta hai — 'ye candidate, sabse paas wale treated village se kitni door hai?' Agar koi treated village hai hi nahi (n=0, jo practically kabhi nahi hoga is project mein, lekin safety ke liye handle kiya gaya), infinity return hoti hai, taaki koi galti se '0 distance' na samjhe. Syntax: n = len(treated_lats) kitne treated villages hain, ginta hai. float("inf") Python mein infinity ki value hai. np.full(n, lon) ek numpy array banata hai, n length ka, jisme har entry lon (candidate ka apna longitude) hai — ye ek hi value ko n baar repeat karta hai, taaki isko treated villages ke poore array ke saath ek saath compare kiya ja sake. GEOD.inv(lon1_array, lat1_array, lon2_array, lat2_array) ek *vectorized* call hai — ek saath sabhi treated villages se candidate ki geodesic doori nikaal deta hai (loop likhne ki zaroorat nahi), aur teen values return karta hai: forward azimuth, backward azimuth, aur distance-in-meters — humein sirf teesri chahiye, isliye pehli do ko _, _ (underscore) se discard kiya jaata hai, jo Python mein 'is value ko ignore karo' ka signal hai. np.min(dists_m) saari doorion mein se sabse chhoti nikaalta hai — matlab sabse paas wala treated village.

EXAMPLE — Ye Function Kaise Bana — Aur Ek Borderline Case Jo Isne Pakda

Entry 24 mein ye function likha gaya, Entry 23 ki 20-duplicate discovery ke direct response mein. Verify karne ke liye (jab tak live Overpass/pyproj access nahi tha sandbox mein), ye check purani, already-fetched 735-village list par manually apply kiya gaya — ek plain-Python Haversine formula ko pyproj ka temporary stand-in bana ke. Result: 50-meter threshold par, 21 matches mile — wahi 20 jo Entry 23 mein already mile the, plus ek naya case: control village 'Jorging' (Upper Siang), treated village 'Tenggo' (Upper Siang) se sirf 40.3 meter door — lekin in dono ke naam mein koi obvious rishta nahi hai (na spelling-variant, na script-collision). Ye ek lower-confidence case hai — ho sakta hai ye bhi wahi settlement ho (bahut paas hai), ya ho sakta hai genuinely do alag, chhote hamlets ho jo village-clusters mein aksar 40 meter jaise close hote hain. Function ne ise flag kiya, lekin decision automatic rehne diya — 50-meter threshold jo bhi pakde, usse control-list se hata diya jaata hai, chahe wo ek clear script-collision ho (Gunji ki tarah) ya ek zyada ambiguous case (Jorging/Tenggo). Ye ek achha reminder hai ki ek accurate check kabhi-kabhi kuch borderline cases bhi pakad leta hai jo shayad galat-positive hon — lekin false-negative (ek asli duplicate miss karna) se zyada safe hai ek thoda conservative

filter rakhna.

overpass_query_for_bbox() — Ek District Ke Liye OSM Se Candidates Mangna

```
def overpass_query_for_bbox(south, west, north, east):
    # retries on 429/502/503/504 with backoff, honors Retry-After if server sends it
    query = f"""
    [out:json][timeout:{OVERPASS_TIMEOUT_S}];
    (
      node["place"~"^(village|hamlet)$"]["name"]({south},{west},{north},{east});
    );
    out body;
    """
```

Purpose: Ye function ek bounding-box (south/west/north/east coordinates) leta hai aur ek Overpass Query Language (QL) string banata hai. Ye query bolti hai: 'is bbox ke andar har node dhundo jiska place tag "village" ya "hamlet" ho, aur jiska koi name ho (naam-less points skip).' Ye exactly wahi filter hai jo genuine, inhabited settlements ko baaki OSM data (roads, buildings, rivers) se alag karta hai. Syntax: F-string (f"..."") triple-quoted, multi-line — curly braces ke andar variables ({south}, {west}, {north}, {east}) seedhe apni values se replace ho jaate hain. ["place"~"^(village|hamlet)\$"] Overpass QL ka apna regex-matching syntax hai — ~ ka matlab hai 'regex se match karo', aur ^ (village|hamlet)\$ ek regex hai jo exactly 'village' ya exactly 'hamlet' match karta hai, kuch aur nahi (jaise 'town' ya 'city' exclude ho jaate hain).

```
last_err = None
for attempt in range(1, MAX_RETRIES + 1):
    try:
        resp = requests.post(
            OVERPASS_URL, data={"data": query}, headers=HEADERS,
            timeout=OVERPASS_TIMEOUT_S + 10,
        )
        resp.raise_for_status()
        return resp.json().get("elements", [])
```

Purpose: Retry-loop shuru hota hai, 1 se 5 tak. Har attempt mein, ye query Overpass ko POST request ki tarah bheji jaati hai (Nominatim ke GET se alag — Overpass lambi queries POST se expect karta hai). Timeout khud ke OVERPASS_TIMEOUT_S (90s) se 10 second zyada rakha gaya hai — taaki agar server ko poore 90 second lage jawaab dene mein, request khud time-out na ho jaaye usi waqt. Agar sab theek raha, response ke JSON se elements list nikaal ke return kar di jaati hai — ye wahi raw OSM candidates hain jo aage filter honge. Syntax: requests.post(url, data={"data": query}, ...) — Overpass API ka apna convention hai ki query ko data dictionary ke andar "data" key mein bhejo. resp.raise_for_status() agar HTTP error code aaya (4xx, 5xx), turant exception uthata hai. resp.json().get("elements", []) safe access hai — agar "elements" key na ho (shouldn't happen, lekin safety ke liye), khaali list milegi crash ke bajaye.

```
except requests.exceptions.HTTPError as e:
    last_err = e
```

```

status = e.response.status_code if e.response is not None else None
if status in RETRYABLE_STATUS_CODES and attempt < MAX_RETRIES:
    wait = RETRY_WAIT_SECONDS * (2 ** (attempt - 1))
    retry_after = e.response.headers.get("Retry-After") if e.response is not None else
None
    if retry_after:
        try:
            wait = max(wait, float(retry_after))
        except ValueError:
            pass
    print(f" {status} error, attempt {attempt}/{MAX_RETRIES} -- waiting {wait:.0f}s
before retry...")
    time.sleep(wait)
    continue
raise

```

Purpose: Ye poore file ka sabse careful-engineered retry-block hai. Agar ek HTTP error aaya (jaise 429 ya 504), pehle uska status-code nikala jaata hai. Agar wo status retryable set mein hai *aur* abhi retries bache hain, toh ek exponential-backoff wait calculate hota hai — 20, fir 40, fir 80, fir 160 second (formula: $20 * (2^{attempt-1})$). Server kabhi-kabhi apne response mein khud bata deta hai kitni der ruko (Retry-After header) — agar wo mila, aur wo calculated wait se zyada hai, uski hi sunee jaati hai (server ki apni advice ko priority). Fir print karke, utni der sleep karke, continue se loop ka agla attempt shuru hota hai. Lekin agar status retryable nahi hai (jaise ek genuine 404), ya retries khatam ho gaye, raise se wahi error dobara uthaayi jaati hai — is baar bina pakde, taaki caller ko pata chale ki genuinely kuch galat hai. Syntax: `except requests.exceptions.HTTPError as e:` specifically HTTP-status-code errors pakadta hai. `e.response.status_code` if `e.response` is not None else None ek conditional expression hai — safe access, agar response object hi missing ho. `2 ** (attempt - 1)` Python ka power operator hai — `attempt=1` par ye 1 hai (2^0), `attempt=2` par 2 (2^1), `attempt=3` par 4 (2^2) — isi se 20, 40, 80, 160 ka pattern banta hai. `e.response.headers.get("Retry-After")` HTTP response headers se ek specific header padhta hai, agar exist kare. Nested `try/except ValueError: pass` — agar server ne Retry-After mein koi non-numeric junk bheja, `float(...)` fail hoga, lekin `pass` se chup-chaap ignore ho jaata hai (calculated wait use hoti rehti hai). `continue` loop ke agle iteration par le jaata hai. `raise` bina kisi argument ke, current exception ko dobara uthata hai.

```

except requests.exceptions.RequestException as e:
    last_err = e
    if attempt < MAX_RETRIES:
        wait = RETRY_WAIT_SECONDS * (2 ** (attempt - 1))
        print(f" network error ({e}), attempt {attempt}/{MAX_RETRIES} -- waiting
{wait:.0f}s before retry...")
        time.sleep(wait)

```

```
        continue
    raise
raise last_err
```

Purpose: Ye doosra except block un errors ke liye hai jo HTTP-status-code ki wajah se nahi, balki pure network-level fail hone ki wajah se hote hain (connection drop, DNS fail, waghera) — is case mein status-code check karne ka sawaal hi nahi uthhta, seedha exponential-backoff retry hota hai. Function ke bilkul end mein, agar poora loop bina success ke khatam ho gaya (jo theoretically nahi hona chahiye, kyunki andar wale blocks khud raise kar dete hain jab retries khatam hon), ek final safety-net raise last_err hai. Syntax: except requests.exceptions.RequestException ek broader exception-class hai jo HTTPError ko bhi include karta hai (isliye ye except block *after* the more specific HTTPError block aata hai — Python pehle-match-hone-wala exception handler use karta hai, order isliye zaroori hai). raise last_err function ke bahar, loop ke bahar hai (indentation dekho) — ye ek extra safety-net hai jo shayad kabhi practically nahi chalega, lekin ise likhna best practice hai taaki function hamesha kisi na kisi tarah 'clearly fail' ho, chup-chaap None return na kare.

EXAMPLE — Ye Poora Retry-Design Kahan Se Aaya

Development Log Entry 13 mein likha hai — jab ye script pehli baar live chalayi gayi, Overpass ka shared server load ke under 429 aur 504 errors de raha tha, khaaskar bade district bounding-boxes par. Ye retry/backoff logic aur per-district checkpointing usi pehli live-run ke response mein add kiye gaye — ek theoretical 'best practice' nahi, balki ek real problem ka real fix. Isi run mein 753 control villages successfully collect hue, sab 14 districts ke across.

`_query_nominatim_once()` — Ek Chhota Helper

```
def _query_nominatim_once(query_str):
    params = {"q": query_str, "format": "json", "polygon_geojson": 1, "limit": 1}
    resp = requests.get(NOMINATIM_URL, params=params, headers=HEADERS,
        timeout=30)
    resp.raise_for_status()
    return resp.json()
```

Purpose: Ye ek simple, single-attempt Nominatim query hai — koi retry-logic khud iske andar nahi hai (retry logic caller, yaani `fetch_district_polygon()`, mein hai). `polygon_geojson: 1` parameter Nominatim ko bolta hai ki sirf coordinate nahi, poora boundary-shape (GeoJSON polygon) bhi bhejo. `limit: 1` sirf sabse best-match wala ek result maangta hai. **Syntax:** Function-naam ke aage underscore (`_query_nominatim_once`) Python convention hai jo batata hai 'ye ek internal helper hai, is file ke bahar directly use karne ke liye nahi bana.' Baaki, ek seedha `requests.get` call, response ko JSON mein convert karke return karna.

`fetch_district_polygon()` — Is File Ka Sabse Historically-Rich Function

Ab hum is file ke sabse complex, aur sabse zyada history-laden function par aa gaye hain. Iska kaam simple sunayi deta hai — ek district ka naam do, uska asli boundary-shape (polygon) wapas milega, taaki bounding-box ke bahar ke candidates ko galti se 'is district ka' na maan liya jaaye. Lekin is function ka current version teen alag generations ka result hai — ek bug jo months tak chhupa raha, tab pakda gaya jab uska symptom (bahut saare bbox-only, unverified rows) itna bada ho gaya ki ignore nahi kiya ja saka.

```
def fetch_district_polygon(district, state):
    # real boundary via Nominatim, so a village just across the district line
    # (still inside our search bbox) doesn't get wrongly counted as in-district.
    # Retries on the same retryable statuses/network errors as the Overpass fetch
    # (a bare single-shot request here used to silently fall back to bbox-only
    # matching on any transient hiccup, not just a genuine "no such place" result --
    # that's what left ~30% of the committed control list with district_verified=False,
    # concentrated in exactly the districts below rather than spread evenly).
    # Also tries a "<district> district, <state>, India" phrasing as a second candidate
    # query: some district names in the source village lists are ambiguous on their own
    # (e.g. Sikkim's "North" district), and appending "district" disambiguates them the
    # way an official gazette reference would.
    query_variants = [f"{district}, {state}, India", f"{district} district, {state}, India"]
```

Purpose: Ye comment block khud ek documented bug-history hai — is function ke docstring-jaise comments seedhe batate hain ki purani version mein kya galat tha (single-shot request, koi retry nahi, koi transient hiccup ho ya genuine 'no such place' ho, dono ko same tarah 'polygon nahi mila' maan liya jaata tha), aur uska concrete impact (~30% control list `district_verified=False` reh gayi, aur ye specific districts mein

concentrated thi, sab jagah evenly spread nahi). Function do alag query-phrasings try karta hai — seedha district-naam, aur 'district' word ke saath — kyunki kuch district-naam akele mein ambiguous hote hain (jaise Sikkim ka 'North' district — akela 'North' bolne se Nominatim confuse ho sakta hai). Syntax: `query_variants = [f"...", f"..."]` ek list hai do f-string queries ki. Comments mein `#` se shuru hone wali lines Python execute nahi karta — ye sirf humare (aur future readers ke) liye hain.

EXAMPLE — Silent Fallback Bug — Kaise 30% Data Chup-Chaap Kharaab Reh Gayi

Development Log Entry 37 mein iski poori kahani hai. Purani version mein, agar Nominatim boundary-query kisi bhi wajah se fail hoti — chahe genuinely 'ye district exist nahi karta' ho, ya sirf ek temporary network hiccup ho — function chup-chaap None return kar deta tha, aur caller (`main()`) us district ko bbox-only matching par fall back kar deta tha, `district_verified=False` flag ke saath. Ye kaam toh karta tha, lekin galat tarike se — genuine transient failures (jo retry se theek ho jaate) permanently 'unverifiable' maan liye jaate the. Result: committed control list ka roughly 30% hissa `district_verified=False` tha, aur teen specific districts mein concentrated tha — random spread nahi, matlab ye systematic tha, luck ka mamla nahi. Fix: is function mein retry-logic add ki gayi (Overpass ke retry pattern jaisi hi), taaki transient failures se genuinely retry ho, aur sirf true 'boundary nahi mili' cases hi None return karein.

```

for query_str in query_variants:
    last_err = None
    for attempt in range(1, MAX_RETRIES + 1):
        try:
            results = _query_nominatim_once(query_str)
            last_err = None
            break
        except requests.exceptions.HTTPError as e:
            last_err = e
            status = e.response.status_code if e.response is not None else None
            if status in RETRYABLE_STATUS_CODES and attempt < MAX_RETRIES:
                wait = RETRY_WAIT_SECONDS * (2 ** (attempt - 1))
                retry_after = e.response.headers.get("Retry-After") if e.response is not None
            else None
                if retry_after:
                    try:
                        wait = max(wait, float(retry_after))
                    except ValueError:
                        pass
                print(f" {status} error on Nominatim query '{query_str}', attempt "
                      f"{attempt}/{MAX_RETRIES} -- waiting {wait:.0f}s before retry...")
                time.sleep(wait)
                continue
            results = None
            break

```

Purpose: Ye ek *nested* loop hai — bahar wala loop dono query-phrasings (seedha naam, 'district' ke saath naam) try karta hai, andar wala loop har phrasing ke liye MAX_RETRIES (5) baar tak koshish karta hai. Ye exactly wahi retry pattern hai jo `overpass_query_for_bbox()` mein dekha — retryable status par wait karke continue, nahi toh `results = None` set karke andar wale loop se break. Syntax: Do levels ka for loop, ek dusre ke andar. `break` andar wale loop ko turant rok deta hai (bahar wale ko nahi) — agar success mila (`results = _query_nominatim_once(...)` bina exception ke chal gaya), `break` se retry-loop khatam ho jaata hai. Agar non-retryable error mila (jaise ek genuine 404), `results = None` set karke bhi break hota hai — is baar 'give up on this phrasing' ke matlab mein.

```

except requests.exceptions.RequestException as e:
    last_err = e
    if attempt < MAX_RETRIES:
        wait = RETRY_WAIT_SECONDS * (2 ** (attempt - 1))
        print(f" network error on Nominatim query '{query_str}' ({e}), attempt "
              f"{attempt}/{MAX_RETRIES} -- waiting {wait:.0f}s before retry...")
        time.sleep(wait)
        continue
    results = None
    break

```

```

if last_err is not None:
    print(f" Nominatim boundary lookup failed for query '{query_str}' after "
          f"{MAX_RETRIES} attempts: {last_err}")
    continue

```

Purpose: Pure network-failures ke liye doosra except, wahi backoff pattern. Andar wala retry-loop khatam hone ke baad, ek check hai: agar last_err abhi bhi set hai (matlab andar wala loop genuinely retries khatam karke fail hua, success se nahi nikla), ek failure-message print hoti hai aur continue se *bahar* wale loop ka agla query-phrasing try hota hai — 'seedha naam' fail hua toh ab 'district' wala phrasing try karo. Syntax: Yahan continue bahar wale for query_str in query_variants: loop par apply hota hai, kyunki ye us loop ke body mein hai, andar wale retry-loop ke bahar (indentation dhyan se dekho — ye line andar wale for attempt... loop se ek level bahar hai).

```

if not results or "geojson" not in results[0]:
    continue
geom = results[0]["geojson"]
if geom.get("type") not in ("Polygon", "MultiPolygon"):
    continue
return shape(geom)

print(f" Nominatim boundary lookup found no usable polygon for {district}, {state} "
      f"after trying {len(query_variants)} query phrasing(s).")
return None

```

Purpose: Agar query successful thi (results mile), toh check hota hai ki result mein genuinely koi geojson shape bhi hai ya nahi (kabhi Nominatim sirf ek point deta hai, koi boundary nahi) — agar nahi, agle phrasing par jao. Agar geometry hai, uska type check hota hai — sirf 'Polygon' ya 'MultiPolygon' hi ek real district-boundary ho sakti hai (kuch aur type, jaise 'Point', ka matlab hai Nominatim ne kuch aur hi match kar diya). Agar sab sahi hai, shape(geom) se GeoJSON ko ek asli Shapely geometry object mein convert karke return kiya jaata hai — is function ka successful exit point. Agar dono phrasings try karne ke baad bhi kuch nahi mila, ek clear message print hokar None return hota hai — is baar genuinely 'boundary nahi mili', na ki 'network fail hua'. Syntax: not results or "geojson" not in results[0] — or se do conditions jud rahi hain: results khaali hai, YA pehle result mein geojson key nahi hai. results[0] list ka pehla item leta hai (Nominatim ne limit: 1 maanga tha, isliye zyada se zyada ek hi result aayega). geom.get("type") not in ("Polygon", "MultiPolygon") — not in ek tuple ke against check karta hai, 'ye value in dono options mein se ek nahi hai.' shape(geom) shapely library ka function hai jo raw GeoJSON dictionary ko ek queryable geometry object mein badalta hai — isi object par aage .contains(...) call hoga.

EXAMPLE — Entry 37 Ke Asli Results — Tawang, Pithoragarh, Aur Sikkim Ka North

Jab ye fixed function, retry-logic aur dual-phrasing ke saath, live chalaya gaya (Entry 37), teen districts jo pehle unverified the, unme se do resolve ho gaye. Tawang (Arunachal Pradesh) ka candidate-count 78 se 67 par aa gaya — matlab 11 candidates jo pehle bbox ke andar the lekin asal district-boundary ke bahar the, ab sahi tarike se exclude hue. Pithoragarh (Uttarakhand) 48 villages par hi raha, lekin ab bbox-matched ki jagah polygon-verified ban gaya — count same raha, lekin confidence badhi. Sikkim ka North district, dono query-phrasings try karne ke baad bhi, koi usable polygon nahi de paaya — ye genuinely Nominatim ka data-gap lagta hai us specific place-name ke liye, na ki koi rate-limit ya network problem (kyunki retry-logic khud successfully chali, sirf dono baar khaali result mila). Overall effect: control list 732 se 721 par aa gayi, aur unverified rows 219 (~30%, teen districts mein) se ghatkar 93 (~13%, sirf Sikkim/North mein) reh gayi.

load_border_union() — India Ki Poori Boundary Ek Shape Mein

```
def load_border_union():
    # unprojected is fine for finding the nearest point; actual distance is geodesic
    (GEOD.inv) below
    boundary = gpd.read_file(BOUNDARY_PATH)
    if "ADM0_A3_1" in boundary.columns:
        india_segments = boundary[(boundary["ADM0_A3_1"] == "IND") |
        (boundary["ADM0_A3_2"] == "IND")]
    else:
        india_segments = boundary[(boundary["ADM0_LEFT"] == "India") |
        (boundary["ADM0_RIGHT"] == "India")]
    return india_segments.union_all()
```

Purpose: Ye function Natural Earth ki poori world-boundary shapefile padhta hai, aur usme se sirf wo segments chunta hai jo India se juda hua koi border-line hain — chahe India uska 'left' side ho ya 'right' side (ek border-line do countries ke beech hoti hai, isliye dono taraf check karna padta hai). Comment mein likha hai ki ye boundary unprojected (seedha lat/lon) form mein rakhi jaati hai — kyunki yahan iska use sirf 'sabse paas ka point kaunsa hai' dhundne ke liye hai, asli doori toh geodesic method se alag se nikaali jaati hai (distance_to_border_km() mein). Syntax: gpd.read_file(path) geopandas ka function hai jo shapefile padhta hai — ek special dataframe deta hai jisme geometry bhi ek column hoti hai. if "ADM0_A3_1" in boundary.columns: check karta hai ki file mein column-naming ka kaunsa version hai (Natural Earth ke alag releases mein column-naming badal chuki hai) — agar naya naming hai, usse filter karo, nahi toh purana naming try karo. | pandas mein 'OR' operator hai jab do boolean-conditions jodni ho row-by-row. .union_all() geopandas ka function hai jo saare selected border-segments ko ek single, combined geometry mein jod deta hai — taaki aage distance nikaalte waqt ek hi object se compare karna pade, sainkdon segments se nahi.

distance_to_border_km() — Asli, Geodesic Doori

```
def distance_to_border_km(lat, lon, border_union_4326):
    pt = Point(lon, lat)
    nearest = nearest_points(pt, border_union_4326)[1]
    _, _, dist_m = GEOD.inv(pt.x, pt.y, nearest.x, nearest.y)
    return dist_m / 1000
```

Purpose: Ye function ek gaon ke lat/lon ko border-line ki doori mein badalta hai — do steps mein. Pehle, un-projected geometry mein hi (fast, approximate) sabse paas ka point border-line par dhundo. Fir, us specific point-pair ke beech ki asli, geodesic doori nikaalo (jo accurate hai). Ye 'pehle nearest-point dhundo, fir uski accurate doori nikaalo' pattern hi is file ka core geodesic-accuracy design hai. Syntax: Point(lon, lat) — shapely mein Point banate waqt order (longitude, latitude) hota hai, na ki (latitude, longitude) — ek common confusion-point, isliye dhyan dena. nearest_points(pt,

border_union_4326) shapely ka function hai jo do geometries ke beech sabse paas ka point-pair deta hai — ek tuple return karta hai (village-side point, border-side point), isliye [1] se doosra (border-side) point liya jaa raha hai. GEOD.inv(x1, y1, x2, y2) wahi geodesic-distance call hai jo nearest_treated_distance_m() mein dekha tha, is baar do single points ke beech (array ke bajaye). dist_m / 1000 meters ko kilometers mein badalta hai.

main() — Poora Orchestrator, Step By Step

Ab tak humne saare 'helper' functions dekhe — har ek apna ek chhota, specific kaam karta hai. `main()` function inhe sabko ek saath jodta hai: treated data load karna, resume-logic decide karna, har district ke liye Overpass query, filtering, aur akhir mein save karna. Ye function poori file ka 'dimaag' hai — is PDF ka baaki hissa isi function ko, tukdo mein, poora samjhaayega.

Step 1 — Treated Villages Load Karna

```
def main():
    treated = pd.read_csv(TREATED_PATH)
    treated = treated[treated["is_core_sample"] == True].copy() # Himachal stays
    illustrative-only
    treated["village_norm"] = treated["village"].apply(normalize_name)

    # full treated set across all districts, so a dupe gets caught no matter which district
    surfaces it
    treated_lats_all = treated["latitude"].to_numpy()
    treated_lons_all = treated["longitude"].to_numpy()
```

Purpose: Sabse pehle, treated villages ki file padhi jaati hai, aur sirf `is_core_sample` wale rows rakhe jaate hain — Himachal Pradesh ke 7 illustrative villages yahan se bahar ho jaate hain, jaisa har analysis file mein consistent tarike se hota aaya hai. Har treated village ka naam normalize karke ek naya column bana diya jaata hai, taaki aage naam-comparison fast ho. Fir, do numpy arrays banaye jaate hain — saare treated villages ke lat/lon, na ki sirf current district ke — kyunki duplicate-check (50-meter proximity) ko poore desh ke treated villages se compare karna hai, sirf apne district se nahi (comment isi ko explain karta hai). Syntax: `treated[treated["is_core_sample"] == True]` boolean-filtering hai — ek True/False column banao (comparison se), fir usse dataframe ko filter karo. `.copy()` ek independent copy banata hai (pandas ka ek 'SettingWithCopyWarning' avoid karne ka standard tareeka, jab aage is dataframe mein naye columns add honge). `.apply(normalize_name)` ek column ke har value par function chala deta hai, ek naya column banata hai. `.to_numpy()` ek pandas column ko ek plain numpy array mein convert karta hai — `GEOD.inv()` jaisi vectorized calls ke liye zaroori hai.

Step 2 — Official Priority Names, Aur Resume Logic Ki Shuruaat

```
official_priority_names_by_state = load_official_priority_village_names()

# resume support: a district only counts as done if ALL its rows are verified,
# otherwise drop its old (possibly unverified/bbox-only) rows and rebuild fresh
existing_rows = []
completed_districts = set()
try:
```

```
existing_df = pd.read_csv(OUT_PATH)
if "district_verified" not in existing_df.columns:
    existing_df["district_verified"] = False
```

Purpose: Pehli line woh function call karti hai jo humne abhi dekha — teeno states ki poori official-name sets load kar leti hai. Fir resume-logic shuru hoti hai: khaali existing_rows list aur khaali completed_districts set se start. Agar purani output file exist karti hai, use padha jaata hai — aur agar usme district_verified column hi nahi hai (bahut purani file ho sakti hai, is column ke aane se pehle ki), ek default False column add kar diya jaata hai, taaki aage ka code crash na kare. Syntax: try: block, kyunki agar OUT_PATH file pehli baar chal rahi script ke liye exist hi nahi karti, pd.read_csv FileNotFoundError degi — jo neeché except mein handle hoga. "district_verified" not in existing_df.columns: ek column-existence check hai.

```
verified_per_district = existing_df.groupby(["state", "district"])["district_verified"].all()
completed_districts = set(verified_per_district[verified_per_district].index)
stale_districts = set(zip(existing_df["state"], existing_df["district"])) -
completed_districts

keep_mask = list(zip(existing_df["state"], existing_df["district"]))
keep_mask = [pair in completed_districts for pair in keep_mask]
existing_rows = existing_df[keep_mask].to_dict("records")
```

Purpose: Ye is file ka sabse important resume-logic hai, aur ye seedhe Entry 14 ke bug-fix ka final, working form hai. Har (state, district) group ke liye, check hota hai: kya saari rows us district ki district_verified = True hain? .all() sirf tabhi True dega jab ek bhi row False na ho. Jo districts poori tarah verified hain, wo completed_districts mein jaate hain — inhe agle run mein completely skip kiya jaayega. Jo districts mein koi bhi row unverified hai, wo stale_districts mein aate hain (set-subtraction se: 'saare districts jo file mein hain, minus jo completed hain'). Fir, sirf completed-districts ki rows rakhi jaati hain (keep_mask) — stale districts ki purani, possibly-unverified rows drop kar di jaati hain, taaki wo fresh se rebuild ho sakein. Syntax: .groupby(["state", "district"])["district_verified"].all() — groupby ek column ke set par grouping karta hai, ["district_verified"] us group ke us column ko select karta hai, .all() har group ke liye 'sab True hain kya' check karta hai — result ek Series hai jiska index (state, district) pairs hai aur values True/False. verified_per_district[verified_per_district] boolean-indexing hai — series ko khud apne True/False values se hi filter karna, sirf True wale rakhna. .index se un survive-karne-wale (state, district) pairs ko nikaal ke set(...) mein daal diya jaata hai. set(zip(col1, col2)) - completed_districts — zip do columns ko pair-pair jodta hai, set-minus operator (-) un pairs ko nikaalta hai jo completed set mein nahi hain. [pair in completed_districts for pair in keep_mask] ek list comprehension hai jo True/False ki ek list banata hai. .to_dict("records") ek dataframe ko dictionaries ki list mein convert karta hai — ek dictionary per row, jo

aage Python list mein easily append ho sakti hai.

```

    if completed_districts:
        print(f'Resuming: {len(completed_districts)} fully-verified district(s) already saved
in {OUT_PATH}, will skip:')
        for state, district in sorted(completed_districts):
            print(f" - {state} / {district}")
    if stale_districts:
        print(f'Re-processing {len(stale_districts)} district(s) with unverified/bbox-only
rows from an earlier run "
            f"(their old rows are dropped and rebuilt with the polygon check below):")
        for state, district in sorted(stale_districts):
            print(f" - {state} / {district}")
except FileNotFoundError:
    pass

```

Purpose: Ye block sirf human-readable status messages print karta hai — kaunse districts skip honge, kaunse rebuild honge. Agar koi bhi purani file exist hi nahi karti (bilkul pehli run), `FileNotFoundError` pakdi jaati hai aur `pass` se chup-chaap aage badha jaata hai — sab kuch khaali (`existing_rows = []`, `completed_districts = set()`) se shuru hoga, jo bilkul sahi hai ek fresh start ke liye. Syntax: `if completed_districts:` — ek khaali set 'falsy' hota hai Python mein, isliye ye check 'kya set mein kuch hai' ka natural tareeka hai. `sorted(...)` output ko consistent, alphabetical order mein print karta hai. `except FileNotFoundError: pass` — 'pass' ek khaali statement hai, 'kuch mat karo, bas aage badho.'

EXAMPLE — Ye Resume-Logic Pehle Kyun Nahi Thi — Entry 14 Ki 'Silently Inert' Kahani

Development Log Entry 14 mein ye poora bug documented hai. Purani version mein, resume-logic sirf ye check karti thi ki (state, district) pair output-file mein *kahin bhi* already present hai — bina ye check kiye ki us district ki rows genuinely boundary-verified thi ya sirf bbox-only, unverified thi. Iska matlab: agar ek district interrupt ho gaya, ya koi purani, kamzor run se aaya, uski bbox-only rows hamesha ke liye 'done' maani jaati rehti — kabhi rebuild hi nahi hoti, chahe `fetch_district_polygon()` mein kitna hi bada fix kyun na ho jaaye aage. Fix: resume-condition ko badla gaya, ki ek district sirf tabhi skip ho jab uski *saari* saved rows polygon-verified confirm hon — baaki sab, purani rows drop karke, fresh se rebuild. Yahi wo exact logic hai jo upar ka code implement karta hai.

Step 3 — Border Union Load, Aur claimed_coords Set

```
print("Loading Natural Earth border geometry (same source as
compute_border_distance.py)...")
border_union = load_border_union()

all_control_rows = list(existing_rows)
# stops the same OSM point getting claimed twice under different districts
# (adjacent districts' 0.6° bboxes overlap on purpose)
claimed_coords = {(round(r["latitude"], 6), round(r["longitude"], 6)) for r in
existing_rows}
```

Purpose: Border-boundary geometry (jo humne `load_border_union()` mein dekha) ek baar load ki jaati hai — poore run mein reuse hogi. `all_control_rows` existing (already-completed districts ki) rows se shuru hoti hai — naye districts ki rows isi list mein aage jud'ti jaayengi. `claimed_coords` ek set hai jo existing rows ke coordinates (6-decimal rounded) rakhta hai — comment batata hai kyun: adjacent districts ki search-bboxes jaan-bujh kar overlap karti hain (generous margin ke liye), toh ek hi OSM point do districts ke candidates mein aa sakta hai. Ye set isse rokta hai. Syntax: `list(existing_rows)` — ek naya list-object banata hai, `existing_rows` ki copy se (original ko accidentally modify hone se bachata hai). Set-comprehension: `{(round(r["latitude"], 6), round(r["longitude"], 6)) for r in existing_rows}` — har existing row ke liye ek tuple (lat, lon) banao, 6 decimals tak round karke, aur sabko ek set mein daal do. Set isliye, kyunki aage sirf 'ye coordinate pehle se claimed hai kya' check karna hai — set-membership check (in) list se bahut fast hota hai.

Step 4 — District-Wise Loop Shuru

```
for (state, district), group in treated.groupby(["state", "district"]):
    if (state, district) in completed_districts:
        print(f"\n--- {state} / {district}: already done and verified, skipping (resume) ---")
        continue
    treated_names = set(group["village_norm"])
    max_dist = group["distance_to_border_km"].max()
    search_radius_deg = 0.6 # ~65km at these latitudes -- generous margin around the
    district's villages

    south = group["latitude"].min() - search_radius_deg
    north = group["latitude"].max() + search_radius_deg
    west = group["longitude"].min() - search_radius_deg
    east = group["longitude"].max() + search_radius_deg
```

Purpose: Poori file ka main loop shuru hota hai — treated villages ko unke (state, district) ke hisaab se groups mein todke, ek-ek district process kiya jaata hai. Agar ye district pehle se 'completed' hai, seedha skip. Nahi toh: is district ke treated villages ke naam ek set mein (fast lookup ke liye), unki sabse zyada border-doori (`max_dist` — ye aage `MAX_DISTANCE_MULTIPLIER` ke saath multiply hokar eligibility-cap banayega), aur ek search bounding-box — treated villages ke min/max lat/lon ke

chaaro taraf 0.6 degree (roughly 65km) ka margin. Syntax: `treated.groupby(["state", "district"])` ek *GroupBy* object deta hai — iterate karte waqt har step par ek tuple (state, district) aur us group ka poora sub-dataframe (group) milta hai. `set(group["village_norm"])` ek column ki saari unique values ka set. `.max()/.min()` ek column ki sabse badi/chhoti value. Char alag lines south/north/west/east banati hain, har ek min/max mein margin jodke ya ghataake.

Step 5 — Overpass Query Aur Naam-Based Exclusion

```
print(f"\n--- {state} / {district}: {len(group)} treated villages, "
      f"querying Overpass for candidates in bbox... ---")
try:
    elements = overpass_query_for_bbox(south, west, north, east)
except Exception as e:
    print(f"Overpass query failed for {district}: {e} -- skipping this district, rerun later.")
    continue
time.sleep(REQUEST_PAUSE_S)

official_names = official_priority_names_by_state.get(state, set())
```

Purpose: Humne pehle dekha tha ki `overpass_query_for_bbox()` khud apna retry handle karti hai — lekin agar sab retries khatam ho jaayein aur phir bhi fail ho, wo function raise karti hai. Yahan uska catch hai: agar poora district-query hi fail ho jaaye, poora district skip ho jaata hai (crash nahi hota, baaki districts process hote rehte hain) — aur ek clear message ke saath ki ise baad mein rerun karna hoga. Query successful hone par, ek politeness-pause, aur is state ke official-priority-names nikal liye jaate hain (agar state ke liye names hi na hon, khaali set — safe default). Syntax: `try/except Exception as e:` — ye sabse broad exception-catch hai jo is file mein dikha (baaki jagah specific exceptions jaise `HTTPError` ya `FileNotFoundError` pakde gaye the). Yahan broad rakhna jaan-bujh kar hai — district-level loop ko kisi bhi tarah ke fail se protect karna hai, taaki ek district ki problem poori script ko na roke. `official_priority_names_by_state.get(state, set())` wahi safe dictionary-access pattern hai jo humne baar-baar dekha — na milne par crash nahi, khaali set milta hai.

```
candidates = []
name_excluded = 0
for el in elements:
    name = el.get("tags", {}).get("name", "")
    if not name:
        continue
    name_norm = normalize_name(name)
    if name_norm in treated_names or name_norm in official_names:
        # matches a treated village name, or an official VVP-I village that never got
        geocoded
        name_excluded += 1
        continue
    lat, lon = el.get("lat"), el.get("lon")
    if lat is None or lon is None:
        continue
    candidates.append({
        "village": name, "district": district, "block": "Unknown",
        "state": state, "latitude": lat, "longitude": lon,
    })
```

Purpose: Overpass se mile raw OSM elements ko candidate-list mein badla jaata hai, teen exclusion-checks ke saath. Pehla: agar koi naam hi nahi hai, skip (khaali-naam wale points useless hain). Doosra: naam ko normalize karke check karo — kya ye kisi treated village ke naam se match karta hai, YA kisi official-lekin-kabhi-geocode-na-hui priority village se match karta hai? Agar haan, ye exclude ho jaata hai (aur counter badhta hai — track karne ke liye kitne exclude hue). Teesra: agar coordinates hi missing hain, skip. Jo bachta hai, ek clean dictionary ki tarah candidates list mein add ho jaata hai — block: "Unknown" isliye kyunki OSM control-villages ke liye block-level administrative naam nahi deta, sirf treated villages ke liye ye official CSV se mila tha. **Syntax:** `el.get("tags", {}).get("name", "")` ek chained safe-access hai — wahi `.get(key, default)` pattern jo PDF 6 mein dekha, do level nested. if not name: khaali string bhi 'falsy' hoti hai Python mein. `name_norm` in `treated_names` or `name_norm` in `official_names` — or se do set-membership checks joded gaye hain, dono treated aur official-priority naam se compare. `name_excluded += 1` ek counter increment karta hai. Dictionary literal candidates list mein append ho raha hai, jisme har key ek future CSV-column banegi.

```
print(f" {len(elements)} OSM place nodes found, {len(candidates)} after removing "
      f"{name_excluded} treated-name / official-priority-name match(es)")

district_polygon = fetch_district_polygon(district, state)
if district_polygon is None:
    print(f" WARNING: could not resolve a real boundary polygon for {state}/{district} --
"
      f"falling back to bbox-only matching. These rows will be marked
district_verified=False; "
      f"candidates outside the true district (but inside the bbox) will NOT be filtered
out.")
    time.sleep(NOMINATIM_PAUSE_S)
```

Purpose: Ek progress-summary print hoti hai (kitne raw nodes mile, kitne naam-exclusion ke baad bache), fir `fetch_district_polygon()` call hoti hai — wahi function jo humne detail mein dekha, apni poori Entry-37 history ke saath. Agar polygon nahi mila (jaise Sikkim ka North district), ek explicit WARNING print hoti hai jo saaf-saaf batati hai ki iska practical matlab kya hai: bbox-only matching hoga, aur bbox ke andar-lekin-asal-district-ke-bahar candidates filter *nahi* honge (kyunki koi polygon hi nahi hai unhe check karne ke liye). **Syntax:** Seedha function-call, None-check, aur conditional print — koi naya syntax nahi, sirf pehle dekhe gaye patterns ka reuse.

Step 6 — Per-Candidate Filtering: Dedup, Contamination, Distance, Polygon

```
dupe_skipped = 0
contamination_skipped = 0
kept = []
for c in candidates:
    coord_key = (round(c["latitude"], 6), round(c["longitude"], 6))
    if coord_key in claimed_coords:
        dupe_skipped += 1
        continue
```

Purpose: Ye poore function ka sabse dense, sabse zyada 'accuracy-critical' loop hai — har ek candidate, char alag checks se guzarta hai, tabhi final list mein jagah paata hai. Pehla check: `claimed_coords` set (jo hum pehle bana chuke) — agar ye exact coordinate kisi *doosre* district ke candidate ne already claim kar liya hai (overlapping bbox ki wajah se), ye skip ho jaata hai. Syntax: `round(c["latitude"], 6)`, `round(c["longitude"], 6)` — 6-decimal rounding ka matlab hai sub-meter precision par match karna (bahut precise). `if coord_key in claimed_coords:` set-membership check, fast operation.

```
nearest_treated_m = nearest_treated_distance_m(
    c["latitude"], c["longitude"], treated_lats_all, treated_lons_all,
)
if nearest_treated_m <= COORD_DUPLICATE_THRESHOLD_M:
    # same physical village as a treated one, just a different name/script in OSM
    contamination_skipped += 1
    continue
```

Purpose: Doosra, aur historically sabse important check: humne pehle dekha `nearest_treated_distance_m()` function, jo *saare* treated villages (sirf current district ke nahi) se doori nikaalta hai. Agar ye doori 50 meter ya kam hai, candidate skip ho jaata hai — ye exactly wo check hai jo Gunji jaisi script-collision duplicates pakadta hai, jo naam-based filter kabhi nahi pakad sakta tha. Syntax: Function-call jiska result seedha ek if condition mein use ho raha hai. `<=` operator, 'less-than-or-equal' — exactly 50 meter bhi exclude hota hai, na ki sirf usse kam.

```
d = distance_to_border_km(c["latitude"], c["longitude"], border_union)
if d > max_dist * MAX_DISTANCE_MULTIPLIER:
    continue
if district_polygon is not None:
    if not district_polygon.contains(Point(c["longitude"], c["latitude"])):
        continue # bbox candidate that isn't actually inside this district
    c["district_verified"] = True
else:
    c["district_verified"] = False
c["distance_to_border_km"] = d
c["coord_key"] = coord_key
kept.append(c)
```

```
# don't claim yet -- might still get cut by the cap below
```

Purpose: Teesra check: candidate ki border-se-doori nikaali jaati hai (geodesic method se, jaisa humne dekha), aur agar ye us district ke treated villages ki max-doori se MAX_DISTANCE_MULTIPLIER (1.5) guna se zyada hai, skip. Chautha check, polygon-containment: agar humare paas is district ka real boundary-polygon hai, check karo ki candidate genuinely uske *andar* hai ya nahi — bbox ke andar hona kaafi nahi, asli district ke andar hona chahiye. Agar polygon nahi mila (Sikkim North jaisa case), ye check skip ho jaata hai aur district_verified = False flag ke saath aage badhta hai — transparent tarike se batate hue ki is row ki district-membership verify nahi hui. Jo bhi survive karta hai, use kept list mein daal diya jaata hai — lekin abhi claimed_coords mein add *nahi* kiya jaata, comment isi ko flag karta hai. Syntax: `d > max_dist * MAX_DISTANCE_MULTIPLIER` ek simple numeric comparison. `district_polygon.contains(Point(lon, lat))` shapely ka geometry-containment check hai — 'kya ye point is polygon ke andar hai?' not ...contains(...) se 'andar nahi hai' check hota hai. Dictionary mein naye keys add karna (`c["district_verified"] = True`) — Python dictionaries mutable hain, existing dictionary mein directly naya key-value jod sakte hain.

EXAMPLE — Claim Karne Mein Dekh Ke Der Kyun — Entry 14 Ki Dedup-Ordering Bug

Ye 'abhi claim mat karo' comment seedha Entry 14 ke doosre bug se juda hai. Purani version mein, ek coordinate ko claimed_coords mein daal diya jaata tha jaise hi wo distance/polygon checks pass karta — cap (agle step mein) apply hone se *pehle*. Problem ye thi: agar District A ke paas bahut saare candidates the (jaise 200), aur cap sirf 60 allow karta (treated-count ka 3 guna), toh baaki 140 candidates bhi already claimed_coords mein claim ho chuke hote — chahe wo file mein kabhi save hi na huye ho. Agar District B ka koi legitimate candidate usi coordinate par (adjacent-bbox-overlap ki wajah se) tha, wo permanently block ho jaata, sirf isliye kyunki District A ne use claim kiya tha, jabki District A ne khud use kabhi istemal hi nahi kiya. Fix: claim ko cap ke *baad* move kar diya gaya, taaki sirf wo coordinates reserve hon jo genuinely file mein save hue.

Step 7 — Cap Apply Karna, Fir Claim Karna, Fir Checkpoint Save

```
cap = len(group) * MAX_CANDIDATES_PER_DISTRICT_MULTIPLIER
kept = sorted(kept, key=lambda c: c["distance_to_border_km"])[:cap]
# only claim what actually survives the cap
for c in kept:
    claimed_coords.add(c.pop("_coord_key"))
    print(f" {len(kept)} candidates kept after distance filter (<= {max_dist *
MAX_DISTANCE_MULTIPLIER:.1f} km), "
          f"district-boundary check, cross-district dedup ({dupe_skipped} duplicate
coordinates skipped), "
          f"treated-coordinate dedup ({contamination_skipped} within
{COORD_DUPLICATE_THRESHOLD_M}m of a "
          f"treated village skipped), and per-district cap ({cap})")
```

Purpose: Cap calculate hoti hai — is district ke treated-count ka 3 guna. kept list ko border-doori ke hisaab se sort kiya jaata hai (sabse paas wale pehle), aur sirf pehle cap candidates rakhe jaate hain — matlab, agar zyada candidates hain cap se, jo sabse paas hain (border ke) wahi priority paate hain. Sirf *ab*, jo bacha (cap ke andar), unke coordinates claimed_coords mein add hote hain — Entry 14 ka fix isi line mein implement hua hai. Ek detailed summary print hoti hai, poore filtering-funnel ki — kitne distance-filter se pass hue, kitne cross-district dedup mein skip hue, kitne treated-contamination mein skip hue, aur final cap kya tha. Syntax: sorted(list, key=lambda c: c["distance_to_border_km"]) — sorted() ek naya, sorted list banata hai, key= parameter batata hai kis property se sort karna hai. lambda c: c["distance_to_border_km"] ek chhota, inline (anonymous) function hai — 'har item c ke liye, uski distance_to_border_km nikaalo, sort isi se karo.' [:cap] slicing hai — pehle cap items lo, baaki chhodo. c.pop("_coord_key") dictionary se ek key *nikaal* ke uski value return karta hai, aur us key ko dictionary se hata bhi deta hai (isliye _coord_key jo humne temporarily store kiya tha, ab final saved row mein nahi rahega — ye sirf internal bookkeeping ke liye tha).

```
all_control_rows.extend(kept)

# Checkpoint after every district so an interruption doesn't lose progress
_save(all_control_rows)
print(f" checkpoint saved ({len(all_control_rows)} control villages so far)")
```

Purpose: Is district ke survive-karne-wale candidates, poore all_control_rows list mein jod diye jaate hain. Fir turant, har district ke baad, ek checkpoint save hoti hai — agar script beech mein interrupt ho jaaye (network down, laptop band), zyada se zyada ek district ka kaam hi dobara karna padega, poora run nahi. Syntax: list.extend(other_list) ek list ke saare items doosri list ke end mein jod deta hai (.append() se alag, jo poori list ko ek hi item ki tarah jod deta — .extend() har element ko individually jodta hai). _save(...) function call — hum ise aage dekhenge, ye ek helper hai jo dataframe banata aur CSV save karta hai.

Step 8 — Loop Khatam, Final Save Aur Summary

```
if not all_control_rows:
    print("\nNo control villages found -- check network access to Overpass API and
rerun.")
    return

control_df = _save(all_control_rows)
print(f"\nSaved {len(control_df)} control villages to {OUT_PATH}")
print(control_df.groupby(["state", "district"]).size().to_string())
```

Purpose: District-loop poora khatam hone ke baad (ya agar poora loop resume ki wajah se skip ho gaya aur kuch naya nahi mila): agar akhir mein bhi koi row nahi hai, ek clear error-message ke saath function return ho jaata hai — crash nahi, bas gracefully ruk jaata hai. Nahi toh, final save hoti hai, aur ek human-readable summary print hoti hai — total villages, aur har (state, district) mein kitne, groupby ke through. **Syntax:** if not all_control_rows: — khaali list bhi 'falsy' hai. return bina kisi value ke, function ko turant khatam kar deta hai. .groupby(["state", "district"]).size() har group mein kitni rows hain, ginta hai. .to_string() is Series ko ek clean, printable text mein badalta hai (bina truncation ke, jo lambi lists ke liye default print behaviour hota hai).

_save() — Ek Chhota Lekin Baar-Baar Chalne Wala Helper

```
def _save(rows):
    control_df = pd.DataFrame(rows)
    control_df["is_core_sample"] = True
    control_df["village_source"] = "control (non-VVP)"
    control_df["village_id"] = range(1, len(control_df) + 1)
    if "district_verified" not in control_df.columns:
        control_df["district_verified"] = False
    control_df = control_df[[
        "village_id", "village", "district", "block", "state", "is_core_sample",
        "latitude", "longitude", "distance_to_border_km", "village_source",
        "district_verified",
    ]]
    control_df.to_csv(OUT_PATH, index=False)
    return control_df
```

Purpose: Ye function checkpoint-save aur final-save, dono jagah reuse hota hai (function-naam ke aage underscore batata hai ye internal helper hai). Rows ki list ko ek dataframe mein badalta hai, teen standard columns add karta hai (sab control villages is_core_sample = True maane jaate hain — matlab, DiD analysis mein shaamil honge; ek label "control (non-VVP)", jo merge_geocoded.py jaisi files mein treated villages se alag pehchaanne mein madad karta hai; aur ek fresh, sequential village_id — dhyan do, ye ID har save par *fir* se banti hai, purani IDs preserve nahi hoti). Agar district_verified column missing hai (edge case safety), default False. Fir columns ka ek fixed order set kiya jaata hai, aur CSV mein save. **Syntax:** pd.DataFrame(rows) dictionaries ki ek list ko seedha ek dataframe mein badal deta hai — har dictionary ek

row banti hai, keys column-naam bante hain. `control_df["col"] = True` saari rows mein ek hi value se naya column bhar deta hai. `range(1, len(control_df) + 1)` 1 se N tak sequential numbers deta hai. `control_df[[col1, col2, ...]]` — double square-brackets se columns ka ek subset, ek specific order mein, select kiya jaata hai. `.to_csv(path, index=False)` file mein save karta hai, `index=False` ka matlab hai pandas ka apna row-number index CSV mein nahi likhna.

EXAMPLE — village_id Har Baar Badalta Hai — Yahi Wo Detail Jisne Entry 37 Mein Do Aur Bugs Pakde

Ye `range(1, len(control_df) + 1)` line chhoti lagti hai, lekin iska ek bada downstream consequence hai. Har baar jab control-list regenerate hoti hai (735 se 732, ya 732 se 721), har village ka `village_id` fresh se 1 se shuru hoke banta hai — same physical village ko har regeneration mein ek alag number mil sakta hai. Development Log Entry 37 mein isi wajah se do aur scripts mein bug mila: `pretreatment_placebo_test.py` aur ek naya `dw_pretreatment_placebo_test.py`, dono control-side data ko `village_id` se join kar rahe the — jo treated villages ke liye safe hai (unka ID stable rehta hai, control-list regenerate hone se nahi badalta), lekin control villages ke liye unsafe, kyunki unka ID har regeneration mein reshuffle ho sakta hai. Dono scripts fix karke coordinate-based join par switch kiya gaya, saath mein ek loud warning agar koi row match na ho. Ye exactly wahi lesson hai jo Entry 15 mein `extract_control_satellite_data.py` ke resume-logic ke saath mila tha (PDF 4 mein dekha) — jab bhi ek list regenerate ho sakti hai, uske saath kaam karne wali har cheez ko coordinate se match karna chahiye, kabhi bhi ek regenerate-hone-wale ID se nahi.

__main__ Block

```
if __name__ == "__main__":  
    main()
```

Purpose: Jab ye file seedhe chalayi jaati hai (command line se python select_control_villages.py), main() function call hota hai, aur poora upar dekha hua process shuru hota hai. Syntax: if __name__ == "__main__": wahi standard Python idiom hai jo PDF 6 mein bhi dekha — sirf tab chalta hai jab file directly run ho, na ki jab isse kisi doosri file se import kiya jaaye.

Poori Kahani, Ek Saath — 753 Se 721 Tak

Ab jab humne poori file line-by-line padh li hai, chalo ek baar poori lineage ko wapas dekhte hain — is baar ye samajh ke saath ki har number-change kis specific code-change se aaya.

Round	Count	Kya Fix Hua
Entry 13	753	Pehli live run — retry/backoff aur checkpointing ke saath
Entry 14 → 15	753 → 735	Resume-logic inert-bug fix + dedup-ordering fix (18 cross-district duplicates hataye)
Entry 23 → 24 → 25	735 → 732	UTM se geodesic distance, 50m contamination filter, official-name filter (20+3 contaminated rows hatayi)
Entry 37	732 → 721	fetch_district_polygon() retry+dual-phrasing fix — Tawang 78→67, Pithoragarh verified, Sikkim North unresolved

Aur ek zaroori reminder, jo PDF 4 mein bhi dekha tha: Entry 25 ka fix (735→732) sirf ek dataset-cleanup nahi tha — usne is poore project ka sabse bada, sabse counter-intuitive result diya. Jo full-year night-lights DiD result pehle 'surviving, significant' lag raha tha (coef +0.1653, p=0.0364), wo contamination hatate hi null ho gaya (coef +0.05046, p=0.30946). Wo result asal mein ek illusion tha — kuch control villages secretly treated villages hi the, apne khud ke against compare ho rahe the. Aur phir Entry 37 mein, jab district-verification poori hui, ek aur, chhota, lekin interesting reversal mila: full-year lights baseline-balance check, jo Entry 25 ke baad resolve ho gaya tha (p=0.090), Entry 37 ke baad dobara imbalanced ban gaya (p=0.03558) — poore project mein ye ekmatra result hai jo do baar reverse hua.

Is PDF Mein Kya Seekha: (1) `select_control_villages.py` district-level matching se control group banati hai, tight distance-band se nahi — ek jaan-bujh kar liya gaya design-choice. (2) `normalize_name()` ek naam-based filter hai jo structurally script-collision duplicates (jaise Gunji ki Devanagari spelling) pakad hi nahi sakta — isiliye ek alag, coordinate-based 50-meter check zaroori hua. (3) `fetch_district_polygon()` ki history sabse lambi hai — ek silent-fallback bug jisne 30% data ko chup-chaap unverified rakha, retry-logic aur dual-query-phrasing se fix hua. (4) `claimed_coords` ko cap-apply-hone-ke-baad hi update karna zaroori hai, warna ek district doosre ko galti se block kar sakta hai. (5) 753 se 721 tak ka poora safar accuracy badhane ka safar tha, kam karne ka nahi — har fix ne data ko zyada trustworthy banaya.

Is 721-village control list se hi PDF 4 mein dekha gaya H4 DiD test banta hai — aur yahi wo file hai jisne poore paper ke headline claim ko sabse zyada baar badla, kyunki isi mein sabse zyada real bugs the, aur unhe hide karne ke bajaye har baar khoja aur fix kiya gaya.

Agla step: core extraction files (NDBI, night-lights, aur unse aage cloud-masking, multi-year, secondary extraction) — PDF 8 se PDF 10 tak, jab bhi Sakshi 'next' bole.